AI651 – Deep Learning for Space, Time and Graphs Assignment 2
Huzaifa Abid (25280009) Fall 2026
Code (public GitHub repository): `https://github.com/ihuzafaone/DLSTG-PA2`

Question 1 — River Forecasting Proposal

Proposal in one paragraph. Every 10 minutes we forecast the water level 60 minutes ahead at S1–S4 (with the intermediate $10, \ldots, 50$-minute levels and an 80 % interval) using a *river-graph forecaster*: per-station causal temporal encoders, followed by *lag-aware attention* that lets every station read only its physically plausible upstream sources (reservoir releases, upstream stations, the monitored tributary T1, rainfall) at the lag implied by a flow-dependent travel time. The unmonitored tributary is a latent node driven by R2 rainfall, and the announced release schedule enters as known future tokens. The system is small ($\approx 1.7 \times 10^5$ parameters, a few milliseconds per forecast) and is built to keep working with missing, late or contradictory data.

1. Architecture diagram

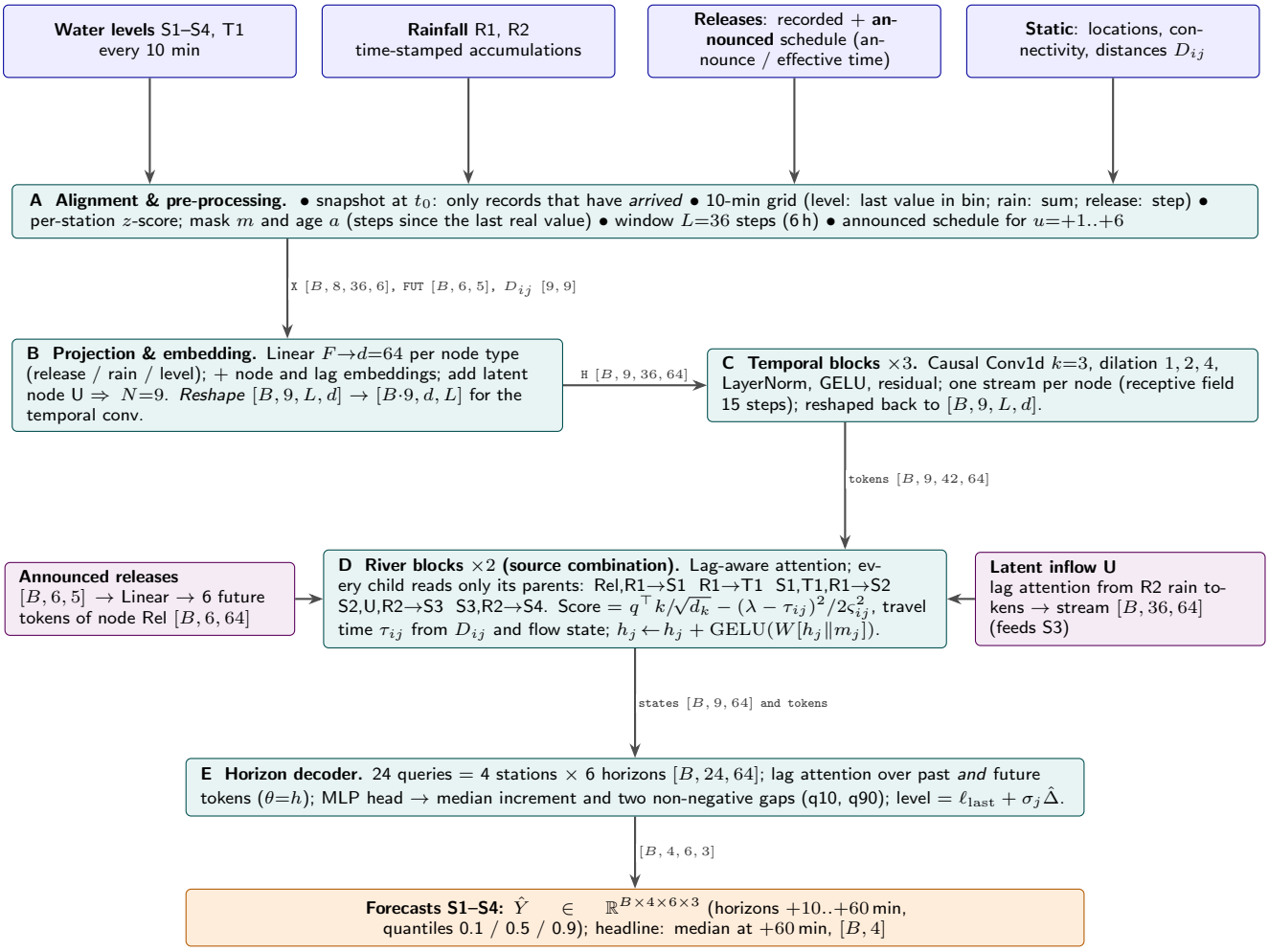


Figure Q1. Architecture. B = batch axis (independent issuance times; B=1 in operation); L=36 past steps; N=9 nodes (Rel, R1, R2, T1, S1, S2, U, S3, S4); token axis 42 = 36 past + 6 future steps; d=64. *Alignment* puts all stations on one clock but does not remove the physical delay between stations (modelled explicitly in D and E); *projection* maps each node type's features to the shared width d; *reshapes* are noted on the blocks. Training: weighted pinball loss on observed targets + celerity prior, chronological split, node/gap dropout and simulated delays.

2. Assumptions

Supplied facts (not assumptions). Four main-river stations S1→S4 in flow order; reservoir above S1; monitored tributary T1 joins between S1 and S2; an *unmonitored* tributary joins between S2 and S3; rain gauges R1, R2 with known locations; levels at S1–S4, T1 every 10 min; rainfall with timestamps; recorded releases plus future schedules announced before each forecast (announcement and effective times); distances along the river; uneven

spacing; travel times may change with conditions. *Not supplied:* travel times, unmonitored inflow, future rain or levels, reporting delays, gauge quality.

	Assumption	Why reasonable	Effect on the design
A1	Records carry a measurement time stamp within ±1 step of the 10-min grid.	Telemetry loggers are clock-driven; jitter of a minute or two is typical.	Re-gridding in block A (last value per bin; rain *summed* per bin).
A2	Levels are stage in metres on a fixed datum per gauge ($\approx \pm 1$–2 cm); rain gauges give accumulations (tipping bucket, ≈ 0.2 mm).	Standard instrumentation; datum offsets between stations do not matter for forecasting changes.	Per-station standardisation μ_i, σ_i; forecasts are *increments* on the last observed level.
A3	Arrival delay is usually 0–2 steps, occasionally up to 1 h; each record has a measurement and an arrival time.	Cellular/radio links are late rather than wrong; telemetry logs normally hold both stamps.	As-of snapshot at t_0; delays simulated in training (§5).
A4	Gaps are mostly short (1–6 steps) with rare long outages; spikes/flat-lines can be flagged by simple QC; failures are more likely during storms.	Usual sensor/communication behaviour; storms damage gauges.	Mask + age channels; node and gap dropout in training; QC-flagged values are treated as missing.
A5	Several years of history are available, with multiple flood rises and release events, and a stable gate-operation regime.	A forecasting service is commissioned from an operating record; "historical" implies years, not days.	Modest model size ($\approx$0.17 M parameters) and shared weights; chronological split; event oversampling.
A6	Dominant flow is one-dimensional downstream; a disturbance moves at a celerity c of order 1–3 m/s ($\approx$ 4–11 km/h) that *rises with stage*; no tide, no backwater from S4's bridge.	Kinematic/diffusive-wave behaviour of mountain rivers; the supplied map shows a free-flowing river.	*Directed* graph (no downstream→upstream links); causal lags $\lambda \geq 0$; lag window ≤ 18 steps (3 h) inside L=36.
A7	Travel times and attenuation depend on flow state; the 60-min horizon is comparable to travel time between neighbouring stations.	Celerity grows with depth; the horizon was chosen by the authority for this river.	Travel time τ_{ij} is *state dependent* with a distance-based prior; spread ς_{ij} grows with distance.
A8	The unmonitored tributary is mostly rain driven (R2 catchment, lag 1–3 h), unregulated; its inflow shows up as a rise at S3 not explained by S2.	It lies between S2 and S3, below the reservoir, and has no gauge or control.	Latent node U fed by R2 rain tokens; S3/S4 decoders also see their own recent trend (it reveals inflow once it arrives).
A9	Point rainfall at R1/R2 is a usable proxy of its sub-catchment's rain; antecedent 24 h/72 h rainfall indicates wetness.	Only two gauges exist; antecedent precipitation is the standard wetness proxy.	Rain nodes R1, R2 with learned lag kernels; antecedent sums as channels; widened intervals (§6).
A10	Announced schedules cover at least the next 60 min; the dam→S1 travel time is < 1 h; actual release usually follows the schedule within $\approx 10\%$ and a step.	A schedule is announced to be acted on; S1 is the first station below the dam.	Future-release tokens on node Rel with *effective* times; deviation channel; fallback in §5(iv).
A11	Channel and gate rules change slowly (months).	Typical for engineered rivers; abrupt changes would be communicated.	Scheduled retraining, drift monitor (§5).
A12	Flood-warning levels exist per station; forecasts support human decisions.	Standard in flood preparedness; output is advice, not automatic gate control.	Flood-weighted loss, quantile (interval) outputs.

3. Forecasting task and data specification

Task. Issuance time t_0 is any instant on the 10-min grid ($t_0 = 10k$ min); a forecast uses only records with arrival time $\leq t_0$ (plus the schedule announced by then). **History window:** L=36 steps, $s \in \{-35, \ldots, 0\}$, i.e. $t_0 - 350, \ldots, t_0$ min (6 h; covers the assumed ≤ 3 h travel plus rainfall response). **Horizon:** H=6 steps, $h \in \{1, \ldots, 6\} = t_0{+}10h$ min; the *service output* is the level at $t_0{+}60$ min (h=6) at S1–S4, the other horizons give the trajectory and training signal. **Output:** $\hat{Y} \in \mathbb{R}^{B \times 4 \times 6 \times 3}$ (stations × horizons × quantiles 0.1/0.5/0.9) converted to metres; B is the batch axis of independent issuance times.

Input / output	Shape	Axes (channels)	Time coverage and availability at t_0
Levels S1–S4, T1	$[B, 5, 36, 6]$	batch, station, step, channels: z-level, Δz (10 min), $z_s - z_{s-6}$ (1 h trend), mask m, age a, QC flag	$t_0-350..t_0$ min at 10-min steps; latest value may be missing/late. Future levels *not* available.
Rainfall R1, R2	$[B, 2, 36, 6]$	batch, gauge, step, ch.: $\log(1+r)$, 1-h sum, 24-h and 72-h antecedent sums, m, a	Same grid; accumulations over each 10-min bin (timestamps snapped). Future rain *not* available.
Recorded releases	$[B, 1, 36, 6]$	batch, node Rel, step, ch.: actual z-release, schedule *as issued then*, deviation, Δ, m, a	Same grid; actual release known up to t_0 (may lag).
Announced future releases	$[B, 6, 5]$	batch, future step u=1..6, ch.: scheduled release, change vs. previous announcement, announcement age, lead $u/6$, mask	Effective times t_0+10u min; announced at or before t_0. Pad/mask if the schedule is shorter.
Static graph data	[9, 9], [9] (no batch axis)	distances D_{ij} (km), parent mask A_{ji}, node-type id	Supplied (locations, connectivity, distances).
Node tensor X (stack of the first three rows)	$[B, 8, 36, 6]$	batch, node (Rel,R1,R2,T1,S1,S2,S3,S4), step, channel (unused = 0)	Latent U is added inside the model $\Rightarrow N$=9.
Output $\hat{Y}$	$[B, 4, 6, 3]$	batch, station (S1..S4), horizon h, quantile	Times t_0+10h min; median at h=6 is the +60-min forecast.
Target Y (training)	$[B, 4, 6]$ + mask	observed level at t_0+10h min	Used only to train/evaluate; missing targets are masked out.

Alignment. *Across sampling times:* every record is assigned to a 10-min bin by its *measurement* time (level: last value in the bin; rain: sum; release: last value held); the bin index s is the same for all nodes, so "step s" means the same wall-clock time everywhere. *Across locations:* the stations are *not* shifted into one another's frame; the physical delay between S1 and S2 is deliberately left in the data and handled by the lag attention (§4). *Units:* levels and releases are z-scored per station, rain is $\log(1+r)$. **Information at issuance:** all data with measurement time $\le t_0$ and arrival time $\le t_0$; the announced schedule for $t_0+10..t_0+60$; static geometry. **Not available:** future rainfall, future levels, actual future releases, records still in transit.

4. Model computations and design justification

Notation. Nodes $V = \{\text{Rel,R1,R2,T1,S1,S2,U,S3,S4}\}$; $P(j)$ = allowed sources of node j (below); $s \in \{-35..0\}$ past step; $\kappa(i)$ node type; $\ell_{i,s}$ raw reading, $z_{i,s} = (\ell_{i,s} - \mu_i)/\sigma_i$; $x_{i,s} \in \mathbb{R}^6$ the channels of the table above; d=64; D_{ij} in km; one step = 1/6 h.

$P(\text{S1}) = \{\text{Rel,R1}\},\ P(\text{T1}) = \{\text{R1}\},\ P(\text{S2}) = \{\text{S1,T1,R1}\},\ P(\text{U}) = \{\text{R2}\},\ P(\text{S3}) = \{\text{S2,U,R2}\},\ P(\text{S4}) = \{\text{S3,R2}\}.$

(i) Embedding (block B)
$$h^0_{i,s} = W_{\kappa(i)}\, x_{i,s} + e_i + p_s \in \mathbb{R}^d \tag{1}$$

(ii) Temporal blocks (block C; $\ell = 1, 2, 3$, causal because future data do not exist, A6; dilation $2^{\ell-1}$; per node)
$$h^\ell_{i,s} = h^{\ell-1}_{i,s} + \text{GELU}\Big(\text{LN}\Big(\sum_{r=0}^{2} W^\ell_r\, h^{\ell-1}_{i,\,s-2^{\ell-1}r}\Big)\Big). \tag{2}$$

(iii) Lag-aware attention (the operator inside blocks D, E and the U stream). A query of node j at target step θ (θ=0 in the encoder, θ=h in the decoder) reads tokens (i, s'), $i \in P(j)$, $s' \le \theta$, with lag $\lambda = \theta - s'$:
$$\tau_{ij} = \frac{6\, D_{ij}}{c_{ij}}, \qquad c_{ij} = \bar{c}_{ij} \exp\big(\tfrac{1}{2} \tanh \phi([h_{j,0} \| h_{i,0}])\big), \qquad \varsigma_{ij} = \tfrac{1}{2} + \text{softplus}(a_{ij}), \tag{3}$$
$$\alpha_{(i,s')} = \underset{(i,s')}{\text{softmax}} \Big(\frac{(W_Q q)^\top (W_K h_{i,s'})}{\sqrt{d_k}} - \frac{(\lambda - \tau_{ij})^2}{2\varsigma^2_{ij}} \Big), \qquad m = W_O \sum_{(i,s')} \alpha_{(i,s')}\, W_V h_{i,s'} \tag{4}$$

(4 heads). $\bar{c}_{ij}$ is a learned per-edge celerity (initialised at 5 km/h, A6), ϕ a small MLP: travel time can shorten at high stage (A7), the spread ς_{ij} grows with distance, and the Gaussian term is only a *prior*: the content score can override it when the data disagree.

(iv) Latent inflow of U (side box of D; A8): for every past step s, $h_{\text{U},s} = \text{Attn}(q = e_\text{U} + p_s;\ \text{R2 tokens},\ \theta{=}s)$ — a learned rainfall→inflow kernel.

(v) River blocks (block D, $\ell = 1, 2$): source combination at θ=0 for the children S1, T1, S2, S3, S4,
$$m_j = \text{Attn}\big(h_{j,0};\ \text{tokens of } P(j)\big), \qquad h_{j,0} \leftarrow h_{j,0} + \text{GELU}\big(W_\ell [\, h_{j,0} \| m_j \,]\big). \tag{4b}$$

One softmax runs jointly over *all* sources of a node, so the model decides how much of S3's next change is upstream water (S2), the unmonitored inflow (U) or direct rain (R2).

(vi) Forecast mapping (block E) for station $j \in \{\text{S1..S4}\}$ and horizon h: query $q_{j,h} = h_{j,0} + e^{\text{hor}}_h$; message $m_{j,h} = \text{Attn}(q_{j,h};\, \theta{=}h)$ over the past tokens and, for node Rel, the 6 future-release tokens;

$$\hat{\Delta}_{j,h} = \text{MLP}\big([q_{j,h} \| m_{j,h} \| h_{j,0}]\big) \in \mathbb{R}^3, \qquad \hat{z}_{j,h,0.5} = z^{\text{last}}_j + \hat{\Delta}^{(1)}_{j,h}, \tag{5}$$
$$\hat{z}_{j,h,0.1} = \hat{z}_{j,h,0.5} - \text{softplus}(\hat{\Delta}^{(2)}_{j,h}), \qquad \hat{z}_{j,h,0.9} = \hat{z}_{j,h,0.5} + \text{softplus}(\hat{\Delta}^{(3)}_{j,h})$$

and $\hat{\ell} = \mu_j + \sigma_j \hat{z}$ in metres; z^{last}_j is the last valid reading (its age is an input, so the head learns to forecast over age+h steps). Monotone quantile construction prevents crossing.

(vii) Training objective (loss in the diagram caption). With $\rho_q(u) = \max(qu, (q-1)u)$ and observation mask o,

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_b \sum_{j=1}^{4} \sum_{h=1}^{6} o_{bjh}\, \omega_{bj} \lambda_h \sum_{q \in \{.1,.5,.9\}} \rho_q\big(z_{bjh} - \hat{z}_{bjh,q}\big) + \beta \sum_{(i,j)} \big(\log \bar{c}_{ij} - \log c^{\text{prior}}_{ij}\big)^2, \tag{6}$$

with $\omega = 1 + 4 \cdot \mathbf{1}[\text{level above warning level}]$ (A12), $\lambda_h = 1$ except $\lambda_6 = 2$ (the service horizon), and c^{prior} a hydraulic prior (initially 5 km/h, replaced by values from a rating/hydraulic model if the authority supplies it, §6). The median (q=0.5) term is a mean-absolute error: robust to spikes and returns a median forecast rather than a mean, which is the right target when the loss is asymmetric or the outcome is multi-modal (e.g. rain or no rain at R2). Chronological train/validation/test split; training windows are oversampled around rises; ensemble of 5 seeds; interval calibrated by conformal adjustment on a rolling validation window.

How each source influences each forecast.

- *S1:* own trend; **releases** (past via lagged attention to Rel; *future announced* via the 6 future tokens at lag $h{-}u \approx \tau_{\text{Rel,S1}}$, A10); R1 rain (A9).
- *S2:* S1 (lag τ_{12}) and **T1** (lag $\tau_{T1,2}$) — two parents of the confluence, combined additively through the shared residual state; R1 rain.
- *S3:* S2 (lag τ_{23}), the **unmonitored inflow** U (R2-driven, A8) and R2 rain; *S4:* S3 and R2. Upstream conditions that are more than h steps away influence S4 only through states already visible at S3.
- *Rainfall:* enters only through the learned lag kernels (rain→runoff delay), modulated by antecedent sums; there is no future rain.

Justification of major choices. (a) *Attention restricted to a directed river graph with a travel-time prior* instead of neighbour convolutions: stations are unevenly spaced and travel times change, so "neighbour" (index distance) and "lag" are *not* uniform; local ST convolutions also have a hard reach (companion notebook: three ST blocks fail beyond three sensors) whereas lag attention gives a direct path from the reservoir or rain gauge to any station at only $\approx 9 \times 42$ tokens (cost $\propto \sum_j |P(j)|\, 42\, d$, negligible). (b) *Dense all-to-all attention* would give the same reach but would also allow physically impossible paths (S4→S1) and more parameters to fit with few floods; the graph mask removes them. (c) *Increments on the last observation* remove datum differences and make persistence the starting point. (d) *Latent node U* lets rainfall explain unmonitored inflow without pretending it is measured.

Physical versus learned. *Imposed:* flow direction (graph), causality ($\lambda{\geq}0$, no future data), positive travel time $\propto$ distance, additive combination at confluences, rain ≥ 0. *Learned:* the state dependence of celerity ϕ, spread ς, the rainfall→inflow kernel, attenuation and the release→level response. Mass conservation is *not* enforced because only stage (not discharge) is measured (§6). *Optional extras not built:* a hydraulic simulator (it would be used to generate extreme-flood training data and to set c^{prior}) and a full Bayesian model; uncertainty is represented by quantile heads + ensemble + conformal calibration.

5. Operation and exceptional conditions

(i) Missing observations. A missing value is not imputed silently: the last valid value is carried in the tensor but flagged (m=0) with its *age* a; attention scores of never-observed tokens are set to $-\infty$ so the softmax renormalises over the remaining sources. During training every window gets random gaps (1–6 steps) and random whole-node dropout (p=0.1), so the network has learned to forecast with any single sensor missing. If the target station itself is missing, eq. (5) forecasts age + h steps from its last value (wider interval). Outputs carry a *degraded* flag with the number of valid inputs; a station silent for > 1 h is reported as low confidence rather than silently continued. *Justification:* masking plus training-time dropout is the only way a model sees the same missingness pattern it will meet in operation.

(ii) Delayed observations. The data store keeps measurement time t_m and arrival time t_a. At issuance only records with $t_a \leq t_0 + 1$ min are used (1-min grace, then no waiting): late records are *missing at that issuance* and are inserted at their correct bin s for all later issuances. Training windows are built point-in-time, with delays resampled from the empirical $t_a - t_m$ distribution, so training and deployment see the same lateness; no hindsight (a record that arrived later is never visible in a training sample issued earlier). Because forecasts repeat every 10 min, a late record simply improves the next one.

(iii) Announced future releases. The schedule is represented as a step function over t_0+10..+60 min from the *latest announcement made at or before* t_0; every future token carries its *effective* time (so a release effective at +30 min is attributed to S1 at +30+$\tau_{\mathrm{Rel,S1}}$ min, never earlier), its change from the previous announcement and its age. If the schedule is missing or shorter than 60 min the uncovered steps are masked and the model falls back on "last actual release held", flagged. Announcements made after t_0 are, of course, unseen.

(iv) Scheduled versus actual releases. History tokens contain both the schedule *as it was issued* and the recorded actual release, and their deviation δ_s; training uses these as-issued schedules, so the network learns $E[\text{actual} \mid \text{schedule}, \delta]$ and its spread (hence a wider interval after deviations), and training schedules are additionally perturbed with the empirical deviation distribution. In operation, if $|\delta|$ exceeds 3× its historical MAD for 3 consecutive steps (schedule evidently not followed) the future-release tokens are *masked* and the model relies on the actual-release history until the deviation ends; an alert is raised to the operator.

Feasibility every 10 minutes. A forecast touches $9 \times 42{=}378$ tokens at width 64; the untrained prototype of exactly this architecture has 1.68×10^5 parameters (≈ 0.7 MB) and runs a forward pass in ≈ 5 ms on one CPU thread (a shape/size check, not a benchmark), i.e. ≈ 25 ms for the 5-member ensemble against a 600 s cycle. The bottleneck is data ingestion, not compute: a rolling buffer keeps the last 6 h plus 72 h of rain totals; each cycle runs ingest → QC → regrid → forecast → publish with a watchdog and a fallback chain (ensemble → single model → persistence with schedule routing → "no forecast" alert). Training (a few hours) is offline; retraining after each flood season or when the drift monitor (rolling skill vs. persistence, input-distribution shift) trips.

6. Uncertainty, limitations, and additional information

Uncertainty / limitation	Impact on the forecast	Response, or what remains unresolved
Unmonitored tributary (S2–S3)	Inflow appears first at S3 and propagates to S4; during rain over its catchment S3/S4 rise "unexplained", mostly on the rising limb.	Latent node U driven by R2 and antecedent rain; S3/S4 heads see their own trend; intervals widen when R2 recently rained. *Unresolved:* magnitude and timing of the inflow are not identifiable without a gauge.
Changing travel times	A fixed lag would shift the predicted rise (too early/late), most at floods; error peaks at hydrograph rises.	State-dependent celerity ϕ, learned spread ς, trained over all regimes; out-of-distribution detector (stage above training range) widens the interval. *Unresolved:* overbank flow and hysteresis in the largest floods.
No future rainfall	Rain falling after t_0 is unknown; at a 1-h horizon this matters most at S1/T1 catchments with short response.	Quantile outputs (80 % band); training on all weather. *Unresolved:* unexpected rainfall within the hour.
Two rain gauges	Convective cells can miss both gauges or be over-represented.	Learned kernels, antecedent sums; interval. *Unresolved* without more spatial coverage.
Release deviations / emergency releases	Actual flow may differ from the schedule (S1 mostly, downstream a lag later).	§5(iv); deviation features; masking of the schedule. *Unresolved:* unannounced emergency releases.
Sensor errors, drift, datum shifts	Biased input ⇒ biased forecast; flat-line = false calm.	QC flags → missing; increments on last value; robust (absolute) loss. *Unresolved:* slow, undetected bias.
Few extreme floods	Skill at the highest levels is least certain, and these matter most.	Flood weighting, oversampling, calibrated intervals, ensemble spread. *Unresolved:* extrapolation beyond the historical record.
Only stage is measured	No discharge ⇒ no explicit mass balance; stage–discharge is non-linear.	Learned implicitly per station; priors from distances only.
Non-stationarity	Channel change, new gate rules degrade skill.	Drift monitor, scheduled retraining; *unresolved* between retrainings.

Additional information requested	How it would help	Usable without it?
R1 Level or discharge gauge on the unmonitored tributary (even a temporary one)	Turns node U from latent to supervised; removes the largest unexplained error at S3/S4.	Yes: latent U (A8) with wider intervals.
R2 Radar / gridded rainfall and 1–3 h rainfall nowcast	Supplies the missing future-rain input and spatial coverage.	Yes: past-rain-only version above.
R3 Rating curves, cross-sections, hydraulic (e.g. 1-D) model	Sets the celerity prior c^{prior}, enables mass-balance priors and *simulated* extreme floods for training.	Yes: priors from distance and 5 km/h start.
R4 Real-time gate telemetry and release operating/emergency rules	Reduces scheduled-vs-actual uncertainty; announces emergency releases earlier.	Yes: deviation features and masking.
R5 Telemetry latency logs, QC/maintenance history	Calibrates the delay simulation and missing-data patterns (A3, A4).	Yes: assumed distributions.
R6 Warning levels per station; more rain gauges / soil moisture	Correct loss weights (A12); better runoff modelling (A9).	Yes: top-5 % levels as proxy; two gauges.

Question 2 — Motion or Memory?

Reading guide. Tables and figures are numbered as in the handout (Table 1–6, Figure 1–6); extra material is labelled S1, S2. Unless stated otherwise, model-performance numbers are on the 1000 test videos (9000–9999), at $1\times$ with $t = 1$; every learned model was trained with seeds 0 and 1 and tables report *mean (seed 0, seed 1)* (per-seed rows are shown); figures and GIFs use the seed-0 model. Errors have three significant figures, scientific notation below 10^{-3}. Code, logs and GIFs: see the repository on page 1 (`src/`, `results/`, `figures/`, `predictions/`).

Hardware and software. NVIDIA GeForce RTX 4060 Laptop GPU (8 GB), 24-thread CPU, WSL2; PyTorch 2.10.0+cu128 (CUDA 12.8, cuDNN defaults, fp32 with TF32 convolutions allowed), torchvision 0.25, OpenCV 4.13, conda env `PA1_DL`. The whole dataset (uint8) lives on the GPU. *Training time* = mean wall-clock seconds per epoch over the 30 epochs (CUDA-synchronised; validation monitoring excluded); *inference time* = median of five timed passes over the 1000 original test windows at batch size 32, batches already on the GPU, `model.eval()` and `torch.no_grad()`. All models were trained one after another on the otherwise idle GPU so that times are comparable.

1. Setup

Table 1: Sanity checks of the transformation and window functions (the same functions every experiment calls). (d), (e): (frame, row, column), 0-based, for $p[7, 0, 10, 20] = 1$. For (a)–(c) `torch.equal` returned `True`.

	Check	Expected	Yours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0.0
(b)	$\max \lvert R^4(x) - x \rvert$	0	0.0
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0.0
(d)	bright pixel of $M(p)$ (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$ (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f)	$1\times$ input / target frames	1–5 / 6–10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g)	$2\times$ input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h)	$3\times$ input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

Videos are split by index (0–7999 / 8000–8999 / 9000–9999), scaled to $[0, 1]$, and windows follow eq. (2) exactly (the same `gather_windows` serves training, evaluation and Table 1). Symmetry $g = 4m + a$ means $M^m R^a$ (rotate first, then mirror); g^{-1} is implemented as "mirror first (if $m = 1$), then R^{-a}".

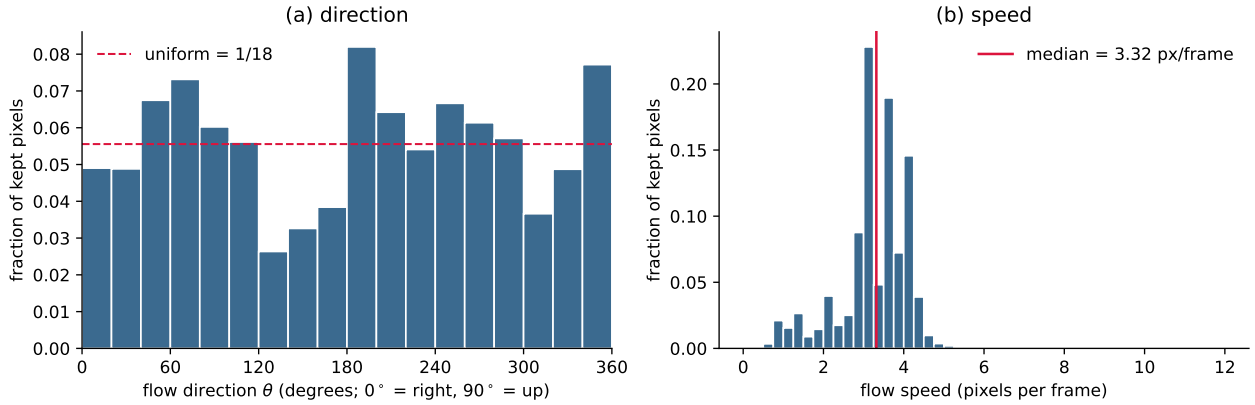


Figure 1: Farneback optical flow on the first 50 training videos, pooled over $50 \times 19 = 950$ frame pairs; kept pixels (intensity > 0.2 in both frames, flow magnitude > 0.5 px): $N = 103{,}050$. (a) direction $\theta = \text{atan2}(-d_y, d_x)$, 18 bins of $20°$ (fraction of kept pixels); the dashed line is the uniform level 1/18. (b) speed in pixels per frame; **median speed** = 3.32 **px/frame** (vertical line).

Q2.1 Yes, mirrored and rotated trajectories are already familiar. Figure 1a is flat within sampling noise: the 18 bins lie between 0.026 and 0.082 around the uniform level 0.056. This is what only 100 independent digit trajectories (50 videos $\times$ 2 digits) can produce: for 100 draws from a perfectly uniform direction law the largest relative deviation of an 18-bin histogram averages 0.88 (95th percentile 1.34, Monte Carlo, 5000 draws), while we observe 0.53. Mirroring sends $\theta \mapsto 180° - \theta$ and a $90°$ rotation sends $\theta \mapsto \theta + 90°$ (4.5 bins). If every training video were mirrored, the histogram would be reflected about the vertical axis; if every video were rotated by $90°$ it would be shifted by 4.5 bins; a flat histogram is unchanged by both, so the transformed directions are as common as the original ones. Quantitatively the total-variation distance between the histogram and its mirror

image is 0.13 (null mean 0.21, 95th percentile 0.31) and to its 90°-rotated copy 0.16 (null 0.21, 0.28): no detectable asymmetry. Elastic bounces also reflect the velocity at the walls, so every trajectory visits mirror-image directions anyway. *Speed:* the median is 3.32 px/frame (inter-quartile range 3.00–3.89, 5–95%: 1.41–4.32; only 0.006% of pixels exceed 8 px/frame). The 2× and 3× videos therefore move about 6.6 and 10.0 px per frame *between the frames the model sees* (8.6 and 13.0 px for the 95th-percentile speed): a range that never occurs at 1×. *Overlap bias:* I recomputed Figure 1 after discarding every frame pair in which the two digits form one blob in either frame (443 of 950 pairs, where flow is unreliable): the median speed becomes 3.24 px/frame, the largest relative deviation of the direction histogram from uniform 0.53 (unchanged) and no pixel exceeds 8 px/frame. The bias is too small to change the answer.

Q2.2

Transformation	Motion statistics	Digit appearance
Mirror M	unchanged: speeds identical, directions $\theta \to 180° - \theta$ (uniform ⇒ same law)	**changed** (a mirrored 2 is not a 2)
Rotation R	unchanged: speeds identical, $\theta \to \theta + 90°$ (same law)	**changed more** (stroke orientation too)
Speed-up $k\times$	**changed**: displacement between seen frames $\times k$, directions unchanged	unchanged (same digits)

Each transformation changes *one* factor and leaves the other fixed: M and R change what the digits look like but keep the family of motions (a constant-velocity trajectory with elastic bounces in a square box is still valid after the symmetries of the square), while speed-up keeps the digits and changes the motion. A model that has learned *how things move* should therefore be indifferent to M and R and should cope with $k\times$ as long as the motion is within its capacity; a failure under M/R points to dependence on appearance, a failure under $k\times$ to unfamiliar motion or to architectural reach. That separation is what makes the three useful as a controlled experiment.

2. Baseline: ConvLSTM

Model A is implemented exactly as specified (pixel-unshuffle(4); two ConvLSTM layers, 64 hidden channels, 3×3 kernels, one convolution for all four gates, no peepholes; 1×1 conv $64 \to 16$, pixel-shuffle, sigmoid; 9-step free-running rollout, no teacher forcing, gradients through the fed-back predictions; states zero-initialised per video). It has **480,784** trainable parameters, as required. Training: Adam (10^{-3}), batch 32, 30 epochs, final-epoch weights, squared error averaged over pixels and available target frames then over videos.

Table 2: Test-set forecast error at 1×, $t = 1$ (1000 test videos): per-horizon MSE, $E^{(5)}$ and trainable parameters.

Predictor	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Params
Model A (mean of seeds 0, 1)	0.0171	0.0203	0.0225	0.0241	0.0258	0.0220	480,784
seed 0	0.0173	0.0206	0.0228	0.0244	0.0260	0.0222	480,784
seed 1	0.0169	0.0201	0.0222	0.0239	0.0256	0.0217	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Q2.3 Reach of model A = 8 original pixels per time step. Follow the longest path inside one step τ to the prediction $\hat{x}_{\tau+1}$. (i) One cell of the folded grid is 4 original pixels; a 3×3 convolution moves information by at most one cell, i.e. 4 px. (ii) The new input u_τ is folded and enters layer 1, whose gates see a 3×3 neighbourhood of it (1 cell); layer 1's *new* hidden state $h_\tau^{(1)}$ is the input of layer 2, whose 3×3 kernel adds one more cell (1 cell); the 1×1 output convolution, pixel-shuffle and sigmoid move nothing. Total 2 cells = 8 px. (iii) The previous state $h_{\tau-1}^{(1)}$ takes the same route through layer 1's recurrent half and layer 2's input: 2 cells; $h_{\tau-1}^{(2)}$ enters only layer 2's recurrent half: 1 cell. The cell state c is combined point-wise and moves nothing. The longest path is therefore $2 \times 4 = \mathbf{8}$ px, *per step*; the recurrence lets state carry information further over several steps, but a digit that jumps farther than 8 px between two frames cannot be connected to its old position inside a single update. I checked this numerically: the gradient of one output pixel of one step with respect to u_τ, $h_{\tau-1}^{(1)}$, $h_{\tau-1}^{(2)}$ is non-zero out to exactly 2, 2 and 1 cells (`src/reach_check.py`), i.e. 8 px.

Q2.4 Versus the references. At 1×, model A (mean of seeds) has $E^{(5)} = 0.0220$, against 0.0426 for Blank (−48%) and 0.0641 for Copy-last (−66%); it is better at every horizon (MSE_1: 0.0171 vs 0.0426 and 0.0492). **Error growth:** MSE rises steadily from 0.0171 (h=1) to 0.0258 (h=5), +51%, i.e. about 0.0022 per horizon, because every predicted frame is fed back and errors compound; Copy-last grows by a similar +49% (0.0492 → 0.0732) and Blank stays flat. Even at h=1 model A is only 2.5× better than Blank: Figure 2 shows that its forecasts are *blurry*, at the right place but without sharp strokes, and they fade and spread with the horizon (the network hedges

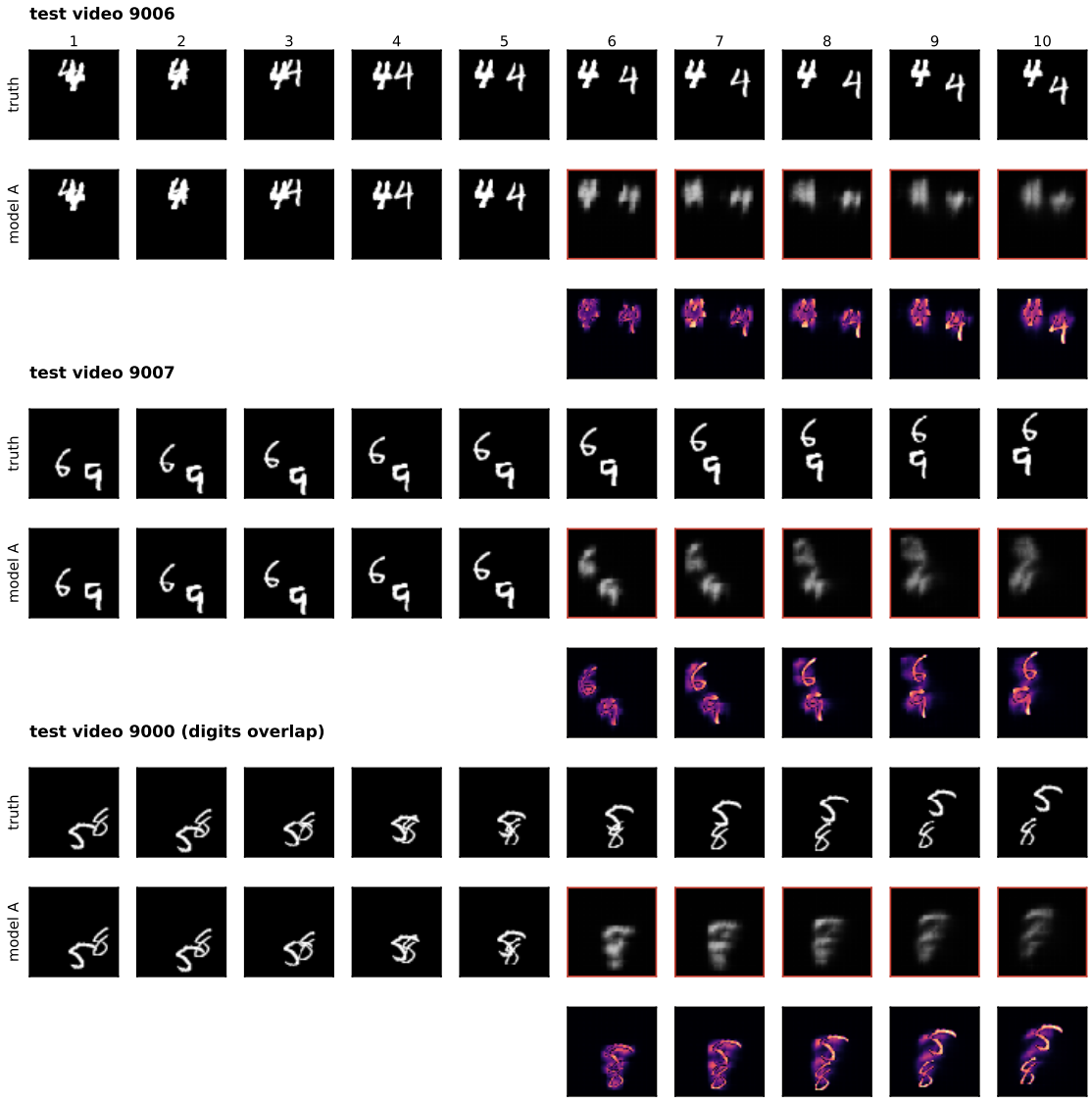


Figure 2: Model A (seed 0) on three test videos: 9006 and 9007 (digits never overlap in frames 6–10) and 9000 (digits overlap). For each: truth (frames 1–10), inputs 1–5 followed by the prediction of frames 6–10 (red frame), and the absolute error.

between possible shapes and positions). **Overlapping digits** (video 9000): the digits cross, and the forecast is a single smeared blob; the error map lights up along both digits' strokes, because the network cannot tell which stroke continues in which direction after the crossing and predicts an average of the possibilities. This is a failure of *identity*, not a large increase of the average MSE: over the 586 test videos whose digits overlap in frames 6–10, model A's $E^{(5)}$ is 0.0216 against 0.0225 for the other 414 (seed-0/1 mean), since overlapping digits cover less area.

Q2.5 Prediction, written before running any "Testing the Baseline" experiment (verbatim from `predictions/Q2.5.md`, committed before `exp_baseline.py` was run). *Written before any Tables 3–4 or Figures 3–5 existed; I disclose that the training logs print the validation $E^{(2)}$ at 2× and 3×, which I had seen (about 0.043 and 0.051 vs 0.022 for $E^{(5)}$ at 1×): the speed part is not fully blind, the mirror/rotation part is.*

> **Mirror test.** Model A will mostly pass: the motion statistics are unchanged (only the digits' appearance changes), so I expect E_M to be somewhat larger than $E^{(5)}$ (about 1.1–1.5×) and C_M clearly non-zero but small (roughly 10–30 % of $E^{(5)}$); both far better than Copy-last. It will not be exactly consistent.
>
> **Rotation test.** Model A will do worse than on the mirror test (E_R about 1.5–3× $E^{(5)}$, C_R larger than C_M): rotation changes the appearance more (stroke orientation changes; mirroring keeps vertical strokes vertical), and a ConvLSTM that partly memorised digit shapes to keep them sharp will blur or distort rotated digits. The motion rule itself should carry over, so I still expect it to stay better than Copy-last.
>
> **Speed.** It will be fine at 1×, degrade noticeably at 2× (never seen in training and the fastest digits already reach the 8 px limit; still better than Copy-last), and break down at 3×, where nearly every digit moves further than the 8 px reach between seen frames: $E^{(2)}$ no better than Copy-last, and visible failures (a digit that stalls near its last seen position, ghosting/blur, or one that vanishes). So the breakdown speed should lie between 2× and 3×, i.e. at an apparent displacement of about 8 px per seen frame.

After the fact (not part of the prediction): mirror—right ($E_M/E^{(5)} = 1.03$, $C_M = 0.08\,E^{(5)}$; even better than guessed). Rotation—ordering right ($C_R > C_M$) but magnitude wrong ($E_R/E^{(5)} = 1.08$, not 1.5–3). Speed—failure type right (stall, blur), but the breakdown starts already at 2× (worse than Blank) rather than between 2× and 3×, and at 3× model A is still better than Copy-last (0.0703 vs 0.0516) although worse than Blank.

3. Testing the Baseline

Table 3: Consistency gaps C_T and transformed errors E_T at 1×, $t = 1$ on the 1000 test videos.

Predictor	$E^{(5)}$	C_M	E_M	C_R	E_R
Model A (mean of seeds 0, 1)	0.0220	0.00177	0.0226	0.00300	0.0237
seed 0	0.0222	0.00162	0.0228	0.00262	0.0237
seed 1	0.0217	0.00192	0.0224	0.00338	0.0237
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

Q2.6 Blank (all zeros) and **Copy-last** are exactly consistent ($C_M = C_R = 0$ in Table 3) because both commute with every pixel permutation: $T(0) = 0$, and copying the last frame commutes with mirroring or rotating it. So C_T measures only whether the predictor treats a scene and its transformed copy *alike*, never whether the prediction is *right*: Blank is perfectly consistent and has $E^{(5)} = 0.0426$, almost twice model A's. Consistency is necessary for a symmetric rule but not sufficient, and a blurrier forecast is more consistent almost by construction, so a small C_T can come from vagueness. E_T, which compares the prediction for the transformed scene with the transformed truth, must always be read next to it. For model A: $C_M = 0.00177$, $E_M = 0.0226$ ($1.03 \times E^{(5)}$), $C_R = 0.00300$, $E_R = 0.0237$ (1.08×) — consistent *and* accurate, both far better than Copy-last (0.0641).

Q2.7 Video 9279, $T = R$ (bottom of Figure 3, rows 3 vs 4; seed-0 model A, 1×). Two digits (9 and 8) overlap in the last input frames and then separate. The truth (row 2) has its ink centroid moving 1.46 px/frame. Row 4, $T(f(X))$ (original orientation), tracks it: centroid speed 1.44 px/frame, the pair separates and drifts. Row 3, $f(T(X))$ (rotated input), *stalls*: the centroid moves only 0.27 px/frame (a fifth), ending ≈ 5 px from where row 4 puts it, and the forecast becomes a horizontally smeared single blob whose total ink grows from 310 (h=1) to 421 (h=5) while the truth stays at 255–273, and whose spread grows from 12.7 to 18.0 px (truth: 9.3 → 12.4). So the model's account changes from "two digits that separate and move on" to "one digit-sized blob that stays put and blurs". The other three videos disagree less: for $T = M$, video 9269, the two rows have the same centroid speed (2.44 vs 2.42 px/frame against the true 3.29) and differ mainly in blur details (centroid gap 1.5 px).

Q2.8 Yes, for model A rotation is harder than mirroring, and the difference comes from appearance, not motion. Table 3: $C_R = 0.00300$ vs $C_M = 0.00177$ (1.7×) and $E_R - E^{(5)} = 0.00174$ vs $E_M - E^{(5)} = 0.00064$ (2.7×). By Q2.1 neither transformation changes the motion statistics: directions are uniform and the speed is unchanged, so a purely motion-based rule would treat both alike. By Q2.2 both change the appearance, rotation more: M flips the digits but keeps vertical strokes vertical and the stroke-orientation statistics roughly the same,

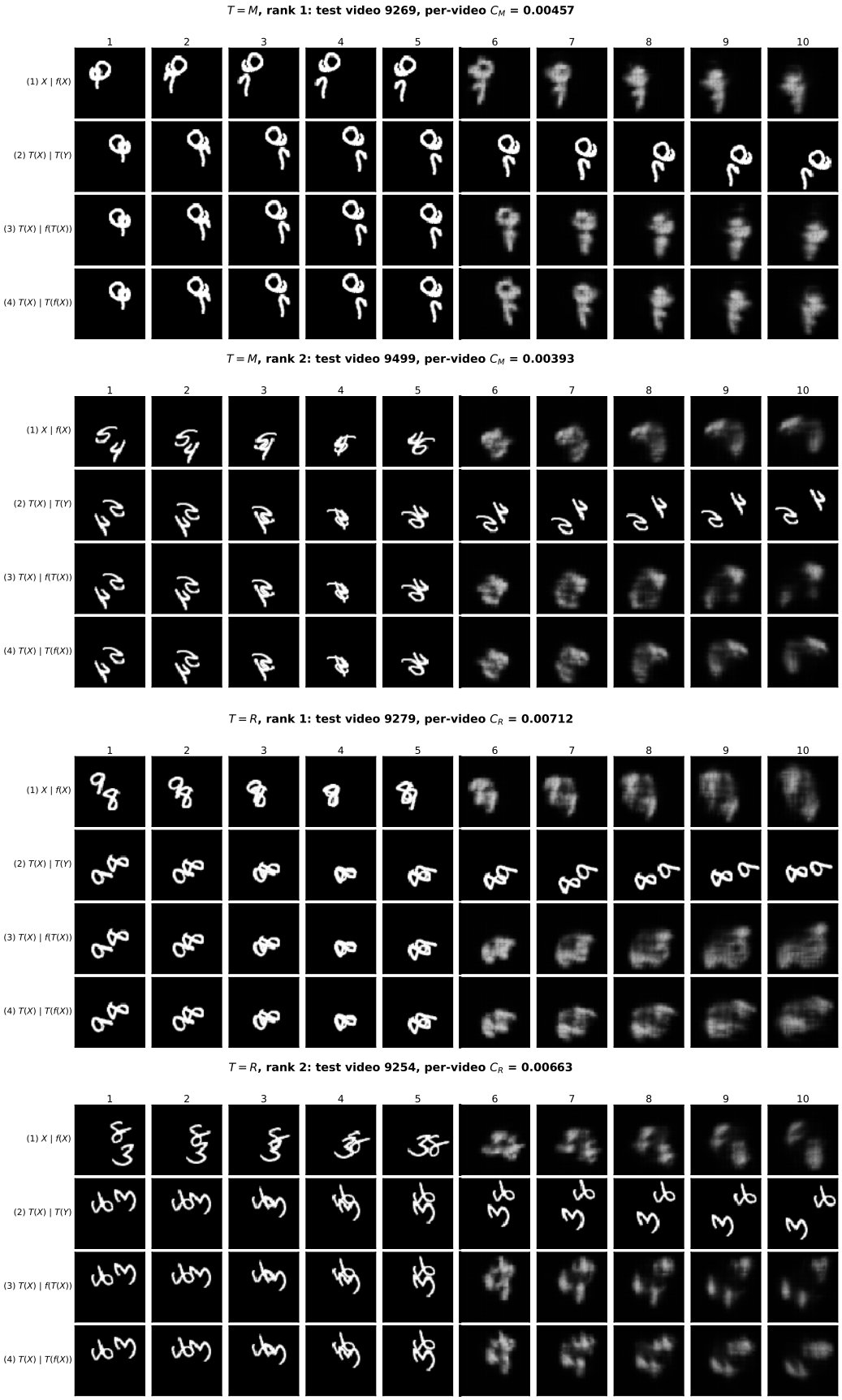


Figure 3: Model A (seed 0), 1×, $t = 1$. Top: $T = M$, the two test videos with the largest per-video C_M (9269, 9499); bottom: $T = R$, largest per-video C_R (9279, 9254). Columns 1–5 are inputs, 6–10 the five future frames; rows (1) $X \mid f(X)$, (2) $T(X) \mid T(Y)$, (3) $T(X) \mid f(T(X))$, (4) $T(X) \mid T(f(X))$. Animated GIFs stepping through the five horizons with the same four rows (last input | horizon h): `figures/gifs/fig3_M_rank1_video9269.gif`, `fig3_M_rank2_video9499.gif`, `fig3_R_rank1_video9279.gif`, `fig3_R_rank2_video9254.gif`.

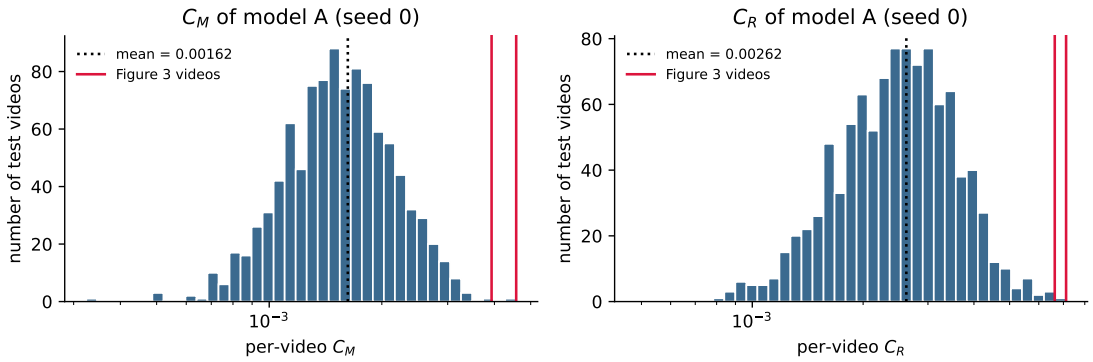


Figure 4: Per-video C_M (left) and C_R (right) of model A (seed 0) over the 1000 test videos (log-scaled axis); red lines: the videos shown in Figure 3; dotted: mean.

whereas R interchanges horizontal and vertical strokes, so rotated digits are further from anything seen in training. The part of the forecast that is carried by digit shape (keeping strokes sharp, resolving overlaps) is the part that degrades, as the stalled, blurred blob in Q2.7 shows. Model B (trained with G) tests this directly: if appearance is the cause, symmetry augmentation should shrink the gap (Table 5). The gap itself is small compared with the speed failure below, so model A's motion rule is largely orientation-agnostic.

Table 4: Speed test on the 1000 test videos ($t = 1$). $E^{(2)}$ uses only the first two horizons (the only ones that exist at 3×).

Predictor	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
Model A (mean of seeds 0, 1)	0.0187	0.0444	0.0516	0.0220	0.0492
seed 0	0.0189	0.0433	0.0508	0.0222	0.0477
seed 1	0.0185	0.0455	0.0524	0.0217	0.0507
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719
Blank	0.0426	0.0425	0.0426	0.0426	0.0425

Q2.9 Where it falls apart. 1× is fine; at 2× the error already jumps to $E^{(2)} = 0.0444$ (2.4× its 1× value 0.0187; better than Copy-last 0.0661 but *worse* than Blank 0.0426) and at 3× it is 0.0516 (Copy-last 0.0703, Blank 0.0426); the $E^{(5)}$ at 2× is 0.0492 against 0.0220 at 1×. Table S1 (row A, in Section 5) measures what goes wrong: from the last input frame to the first forecast, a digit's centroid truly moves 3.6, 7.1, 10.4 px at 1×, 2×, 3×; model A moves its predicted digit by 3.3–3.4 (95% of the truth), 2.5 (39%) and 1.8–2.3 px (19–25%). So *it does not agree with my reach estimate*: with 8 px reach, 2× (true displacement 7.1 px for the median digit, < 8) should still work for most digits, yet the failure already starts there, and by 3× ($10.4 > 8$) it is nearly total. What the estimate missed: the reach is a *maximum* possible path, not what the network has learned to use. Trained only on displacements of about 3–4 px, the outer cells of its kernels (those that would carry a 7 px jump) have never been needed, so the effective reach is smaller than the nominal one, and 2× is also simply unfamiliar. **What the failure looks like (Figure 5):** *stalling* and *blur*: at 2× and 3× the digits appear washed out and close to the position where they were last seen, with a faint trailing smear toward the true position, but they do not move k× as far. I see no duplicated digit and no complete vanishing (intensity is lower but present). A stalled digit means the network failed to link the old position held in its state with the new position (it falls back on "no motion"); a blur means it spreads probability over positions it cannot distinguish, as expected under squared error.

Q2.10 No, not from these results alone. Model A has a single training speed *and* a single, small reach, so the two explanations predict the same thing, a failure at 2× and 3×: "fast motion never appeared in training" and "it is beyond the model's reach" are confounded. The displacement numbers only hint: 2× (7.1 px, below the nominal reach) fails, favouring unfamiliarity at least there, but the effective reach is unknown. **An experiment that separates them:** train the same architecture on all three speeds (model C: familiarity restored, reach unchanged) and a larger-reach architecture (model D) on the same data, and evaluate each at 2× and 3×: if C fixes a speed, that failure was unfamiliar motion; if C still fails where D succeeds, it was reach. A cleaner variant is the *reach-only* control, a larger-reach model trained at 1× only: if it still fails at 2×, unfamiliarity is the cause, even though the displacement is now within reach.

4. What Can Data Fix?

Models B and C each change one thing relative to A; model D changes only the architecture relative to C; D+B (optional) adds symmetry augmentation to D. All were trained with the protocol of *Training and evaluation* (seeds 0 and 1). Symmetry augmentation draws $g \in G$ uniformly per training window (including the identity)



Figure 5: Model A (seed 0), test video 9021 (digits do not overlap in any displayed true frame): five input frames, first two true future frames, and the first two predicted frames (red) at 1×, 2×, 3× ($t = 1$; frame numbers above each column).

and applies it to input and target; speed augmentation draws $k \in \{1, 2, 3\}$ and then t uniformly from the allowed starts (14/8/2).

Q2.11 Prediction, written before training B, C or D (verbatim from `predictions/Q2.11.md`, committed before training B, C or D; no candidate design of D had been trained either).

> Testing the Baseline showed that model A is almost consistent under mirror and rotation (E_M and E_R within about 8 % of $E^{(5)}$, C_M and C_R about 8 % and 14 % of $E^{(5)}$), but that at 2× and 3× its $E^{(2)}$ (0.044, 0.052) is more than twice its 1× value and worse than the all-zero Blank predictor; its reach (8 px) is below the apparent displacement at 3× (about 10 px) and borderline at 2× (about 6.6 px typical, 8+ px for the fastest digits).
>
> **Prediction (three sentences).** Model B (symmetry augmentation) will lower C_M, C_R, E_M and E_R only slightly, because there is little appearance gap left to close, and will not help at 2× or 3×. Model C (speed augmentation, same reach) will fix most of the 2× failure (unfamiliar motion) but will still fail clearly at 3×, where the apparent displacement exceeds its 8 px reach. Model D (reach 16 px, with speed augmentation) will fix 3× as well, at little or no cost in 1× accuracy.

After the fact: B—right. C—right at 2×, **wrong at** 3×: C also fixes most of the 3× failure ($E^{(2)}$ 0.0516 → 0.0332). D—it fixes 3×, but only slightly better than C (0.0320), and not "at little cost" at 1×: the cost ($E^{(5)}$ 0.0220 → 0.0267) is already paid by C. The reach I estimated was *not* the main bottleneck (see Q2.13).

Table 5: Models A–D (and optional D+B) on the 1000 test videos, both seeds. $E^{(5)}$, C_T, E_T at 1×, $t = 1$; $E^{(2)}$ at 1×, 2×, 3×. Times: see hardware in the text.

Model	$E^{(5)}$	C_M	E_M	C_R	E_R	$E^{(2)}_{1\times}$	$E^{(2)}_{2\times}$	$E^{(2)}_{3\times}$	Params	s/epoch	Infer. (s)
A baseline (mean)	0.0220	0.00177	0.0226	0.00300	0.0237	0.0187	0.0444	0.0516	480,784	13.6	0.586
seed 0	0.0222	0.00162	0.0228	0.00262	0.0237	0.0189	0.0433	0.0508	480,784	13.6	0.596
seed 1	0.0217	0.00192	0.0224	0.00338	0.0237	0.0185	0.0455	0.0524	480,784	13.5	0.576
B +sym. aug. (mean)	0.0223	0.00128	0.0223	0.00153	0.0223	0.0187	0.0447	0.0509	480,784	23.1	0.573
seed 0	0.0230	0.00128	0.0231	0.00163	0.0229	0.0192	0.0450	0.0520	480,784	23.1	0.575
seed 1	0.0216	0.00128	0.0216	0.00144	0.0216	0.0182	0.0444	0.0498	480,784	23.1	0.572
C +speed aug. (mean)	0.0269	0.00109	0.0271	0.00206	0.0279	0.0234	0.0289	0.0332	480,784	14.0	0.574
seed 0	0.0274	8.76e-04	0.0277	0.00166	0.0283	0.0240	0.0295	0.0335	480,784	14.0	0.568
seed 1	0.0263	0.00130	0.0265	0.00245	0.0274	0.0228	0.0283	0.0330	480,784	14.0	0.579
D larger reach (mean)	0.0267	0.00109	0.0269	0.00235	0.0278	0.0238	0.0282	0.0320	603,088	20.8	0.648
seed 0	0.0267	0.00129	0.0269	0.00272	0.0279	0.0238	0.0282	0.0320	603,088	18.8	0.650
seed 1	0.0267	8.93e-04	0.0269	0.00199	0.0278	0.0239	0.0283	0.0319	603,088	22.8	0.646
D+B (mean)	0.0276	4.55e-04	0.0277	4.78e-04	0.0277	0.0247	0.0295	0.0330	603,088	33.0	0.657
seed 0	0.0275	5.96e-04	0.0275	5.68e-04	0.0276	0.0246	0.0295	0.0328	603,088	35.7	0.651
seed 1	0.0278	3.15e-04	0.0278	3.88e-04	0.0278	0.0249	0.0296	0.0332	603,088	30.2	0.663
Copy-last	0.0641	0	0.0641	0	0.0641	0.0548	0.0661	0.0703	0	–	1.07e-04

Timing note. Times are the logged means over the 30 epochs and the timed inference of the final weights. Some logged epochs of D (seed 0) and D+B (seed 1) were up to 2× faster than the rest (GPU/WSL power-state or clock changes during that period), so I also re-timed all five architectures in a controlled run (idle GPU, seed 0, mean of epochs 2–4): A 10.9 s, B 17.1 s, C 10.7 s, D 14.5 s, D+B 20.5 s per epoch; the ordering (augmentation overhead of the per-sample symmetry loop; D's dilated, wider layers) is the same as in the table. The inference times of A, B, C are equal (≈ 0.58 s for 1000 videos) because they share the architecture; D's is ≈ 0.65 s.

Q2.12 Model D is the same 9-step free-running ConvLSTM as A (patch 4, two layers, input/output shapes, no teacher forcing, same training protocol), with a different spatial configuration: hidden width 72 (instead of 64) and a *dilated* second layer: layer 1 is a 3×3 convolution (dilation 1), layer 2 a 3×3 convolution with **dilation 2** (zero padding 2 keeps the 16×16 size). It has 603,088 trainable parameters (1.25× A; limit 721,176). The design was chosen on the *validation split only* from six candidates (dilated, 5×5-kernel, wider and patch-8 variants; all within the parameter limit and with reach > 8 px), each trained for 12 epochs with the C protocol (seed 0) and ranked by the pre-declared score mean($E^{(5)}$@1×, $E^{(2)}$@2×, $E^{(2)}$@3×) (`results/d_design.json`); this one scored best (0.0316 vs. 0.0339 for the runner-up). Caveat: three candidates (d2, d4, d5 in `results/d_design.json`) were still on the initial loss plateau after 12 epochs, so the choice partly reflects how quickly a design starts learning.

Reach, same method as Q2.3. Radius of a dilated 3×3 convolution $= d \cdot (k-1)/2$ cells: layer 1 1 cell, layer 2 2 cells. Longest paths in one step: new input $u_\tau \to$ layer 1 (1) $\to$ layer 2 (2) = 3 cells; $h^{(1)}_{\tau-1} \to$ layer 1 (1) $\to$ layer 2 (2) = 3 cells; $h^{(2)}_{\tau-1} \to$ layer 2 recurrent (2) = 2 cells; the 1×1 convolution, pixel-shuffle and sigmoid add nothing. With one cell = 4 px the reach is $3 \times 4 =$ **12** px, strictly larger than A's 8 px (the offsets $\{-1, 0, 1\} + \{-2, 0, 2\}$ cover $-3..3$ without gaps, so no "checkerboard" blind spots). Verified numerically with autograd (`src/reach_check.py`: 3, 3, 2 cells ⇒ 12 px). Compared with the displacements of Q2.9: 12 > 10.4 px, so D can reach the median digit even at 3× (A could not).

Q2.13 Labels (Table 5, Table S1; means of seeds).

- *Appearance* (mirror/rotation, 1×): small in A and removed by B. $E_R - E^{(5)}$: A 0.0017 (A: $E_R = 0.0237$) → B −0.0001 ($E_R = 0.0223$); C_R: 0.00300 → 0.00153; C_M: 0.00177 → 0.00128. Models without symmetry

augmentation (C, D) keep a gap ($E_R - E^{(5)} = 0.0010$–0.0012), B closes it, which is why I attribute it to appearance.

- *Unfamiliar motion* (2× and 3×): B does not help ($E^{(2)}$ 0.0447, 0.0509 vs A's 0.0444, 0.0516) but C does: $E^{(2)}_{2\times}$ 0.0444 → 0.0289 (−35%), $E^{(2)}_{3\times}$ 0.0516 → 0.0332 (−36%), and the digits' displacement at 2× rises from 39% (A) to 84% of the truth (Table S1), at 3× from 22% to 72%. Same architecture, same reach, only the training speeds changed: this is the dominant cause, at both speeds.
- *Architectural reach*: D versus C changes only the architecture: $E^{(2)}_{3\times}$ 0.0332 → 0.0320 (−3.7%, lower in both seeds: C 0.0335, 0.0330; D 0.0320, 0.0319), $E^{(2)}_{2\times}$ 0.0289 → 0.0282 (−2.4%); displacement at 3× 72% → 77%. A real but small contribution, mainly at 3×, where the true displacement (10.4 px) exceeds A's reach.

If C still did badly at 3× while D did well that would show that the failure was *architectural*: more training data cannot carry information further than the architecture allows, because the output at a pixel is a function of a bounded set of input cells (the gradient with respect to anything beyond the reach is exactly zero, as checked numerically) and training can only change how the available paths are weighted, never create new ones. Here it did not happen: C already reaches 72% of the true 3× displacement with its nominal 8 px reach, so, as Q2.9 anticipated, the nominal reach overstates the true limitation once the network has seen fast motion. That *I predicted otherwise in Q2.11 was wrong.*

Q2.14 Yes, there is a cost at 1×, and it comes from speed augmentation, not from the architecture. $E^{(5)}$ at 1×: A 0.0220, B 0.0223 (+1.7%, within the seed spread 0.0216–0.0230), C 0.0269 (+22%), D 0.0267 (+22%); $E^{(2)}$ at 1×: A 0.0187, C 0.0234 (+25%), D 0.0238 (+27%). The network now spends its fixed capacity and 30 epochs on three speeds, so it is less sharp at the one it was good at. **Worth it?** For a forecaster that must work at several speeds, yes: $-36\%/-38\%$ error at $2\times/3\times$ (D) for +22% at 1×; A at 2× and 3× is worse than predicting a blank frame (0.0426), whereas C and D beat Blank by about 32% at 2× and 22–25% at 3×. For a 1×-only use case it is not worth it. B is essentially free in accuracy and improves consistency (Q2.15), so yes for B. D costs ≈ 1.1× inference time over C and ≈ 1.4× epoch time, for −3 to −4% error at 3× and no gain at 1×.

5. The Final Model

Why $\bar{f}$ has $C_M = C_R = 0$ exactly. Let $T \in \{M, R\} \subset G$ and f any forecaster. G is a group, so for fixed T the map $g \mapsto h = gT$ is a bijection of G, and $g^{-1} = Th^{-1}$. Then

$$\bar{f}(T(X)) = \tfrac{1}{8}\sum_g g^{-1}\big(f(gT(X))\big) = \tfrac{1}{8}\sum_h T\,h^{-1}\big(f(h(X))\big) = T\Big(\tfrac{1}{8}\sum_h h^{-1}(f(h(X)))\Big) = T(\bar{f}(X)),$$

because T only permutes pixels (a linear map that commutes with the average). Hence $C_T = \mathrm{MSE}(\bar{f}(T(X)), T(\bar{f}(X))) = 0$ for every f, and also $E_T = \mathrm{MSE}(T\bar{f}(X), T(Y)) = E^{(5)}$. In floating point, the inputs $gT(X)$ and $h(X)$ are bit-wise identical (pure data movement), so every call of f gives identical outputs, and only the order of the eight summands differs; I accumulate in float64, so the measured C_M, C_R are 0 (to the printed precision: exactly 0.00).

Choice of the final model. The final model is **D wrapped as a symmetrised forecaster**. I also trained D+B (optional) and chose between them on the *validation* split with the wrapped forecasters: $E^{(5)}$ 0.0264 (sym. D) vs 0.0272 (sym. D+B) (mean of seeds): D is better, and it is also cheaper to train, so the final model is sym. D.

Table 6: Symmetrised forecasters $\bar{f}$ (eq. 7) on the test set. Training time is that of the underlying learned model; inference includes all 8 transformations, forward passes, inverse transformations and averaging.

Model	$E^{(5)}$	C_M	E_M	C_R	E_R	$E^{(2)}_{1\times}$	$E^{(2)}_{2\times}$	$E^{(2)}_{3\times}$	Params	s/epoch	Infer. (s)
sym. B (mean)	0.0217	0.00e+00	0.0217	0.00e+00	0.0217	0.0183	0.0440	0.0500	480,784	23.1	2.60
seed 0	0.0223	0.00e+00	0.0223	0.00e+00	0.0223	0.0187	0.0443	0.0511	480,784	23.1	2.77
seed 1	0.0210	0.00e+00	0.0210	0.00e+00	0.0210	0.0178	0.0437	0.0488	480,784	23.1	2.44
sym. D (mean)	0.0265	0.00e+00	0.0265	0.00e+00	0.0265	0.0237	0.0282	0.0318	603,088	20.8	3.13
seed 0	0.0264	0.00e+00	0.0264	0.00e+00	0.0264	0.0236	0.0281	0.0319	603,088	18.8	3.08
seed 1	0.0267	0.00e+00	0.0267	0.00e+00	0.0267	0.0239	0.0282	0.0318	603,088	22.8	3.19
sym. D+B (mean)	0.0275	0.00e+00	0.0275	0.00e+00	0.0275	0.0246	0.0294	0.0329	603,088	33.0	3.14
seed 0	0.0273	0.00e+00	0.0273	0.00e+00	0.0273	0.0244	0.0294	0.0327	603,088	35.7	3.09
seed 1	0.0276	0.00e+00	0.0276	0.00e+00	0.0276	0.0248	0.0294	0.0331	603,088	30.2	3.20

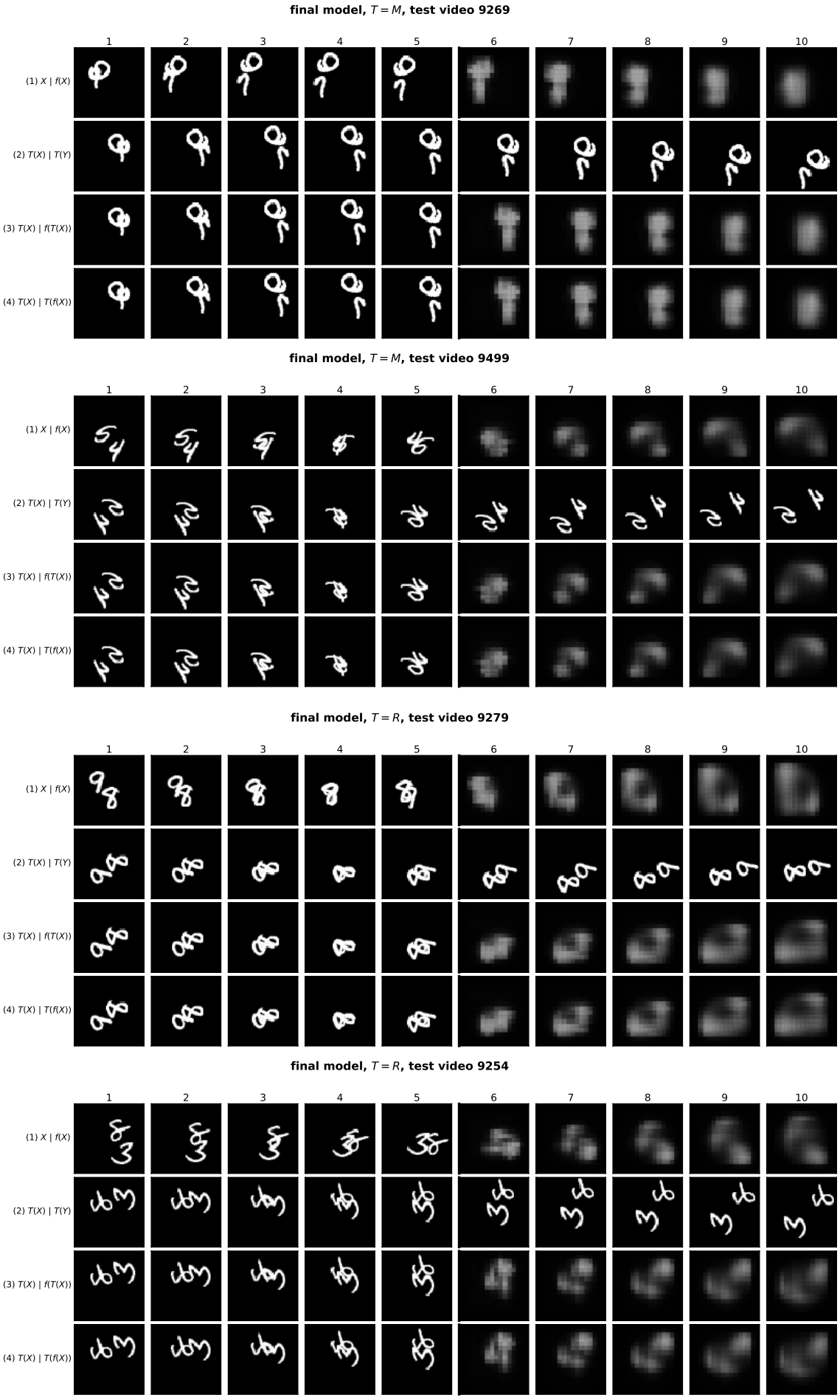


Figure 6(a). Figure 3 repeated with the final model (symmetrised D, seed 0) on exactly the same four test videos (top $T = M$: 9269, 9499; bottom $T = R$: 9279, 9254); rows as in Figure 3. Rows 3 and 4 are identical by construction. GIFs: `figures/gifs/fig6a_M_rank1_video9269.gif`, `fig6a_M_rank2_video9499.gif`, `fig6a_R_rank1_video9279.gif`, `fig6a_R_rank2_video9254.gif`.



Figure 6(b). Figure 5 repeated with the final model (symmetrised D, seed 0) on exactly the same test video (9021): inputs, first two true and predicted future frames at 1×, 2×, 3×.

Table S1 (supplementary). Median centroid displacement (px) of a digit from the last input frame to the first forecast frame at 1×, 2×, 3× (digits that form separate blobs in both frames; mean over seeds 0, 1 of each model's median; in brackets: median predicted/true per digit). Truth: 3.61, 7.07, 10.44 px.

Model	1×	2×	3×
A (baseline)	3.36 (96%)	2.49 (39%)	2.05 (22%)
B (+sym. aug.)	3.40 (97%)	2.64 (42%)	2.56 (27%)
C (+speed aug.)	3.23 (93%)	5.43 (84%)	6.84 (72%)
D (reach 12 px)	3.32 (96%)	5.73 (87%)	7.26 (77%)
final: sym. D	3.40 (97%)	5.80 (88%)	7.15 (76%)

Table S2 (supplementary). $E^{(5)}$ (mean over seeds, 1×, $t = 1$) on forward and on *time-reversed* test videos, and on the 586 test videos whose digits overlap in frames 6–10 versus the other 414.

Model	forward	time-reversed	overlap in 6–10	no overlap
A (baseline)	0.0220	0.0219	0.0216	0.0225
B (+sym. aug.)	0.0223	0.0222	0.0219	0.0230
C (+speed aug.)	0.0269	0.0266	0.0259	0.0282
D (reach 12 px)	0.0267	0.0265	0.0257	0.0280
final: sym. D	0.0265	0.0264	0.0255	0.0280

Q2.15 Learned (B) versus built-in (symmetrised). *Consistency:* B reduces the gaps (C_M 0.00177 → 0.00128, C_R 0.00300 → 0.00153) but not to zero; the wrapped models have $C_M = C_R = 0$ *exactly*, for any f (proof above). Augmentation only *encourages* the symmetry on the data it saw; averaging *imposes* it, so only the built-in version is exact. *Accuracy on original clips (1×):* sym. B $E^{(5)} = 0.0217$ vs B 0.0223 (−2.8%); sym. D 0.0265 vs D 0.0267 (−0.6%); on transformed clips, $E_M = E_R = E^{(5)}$ for the wrapped models by the identity above (B: 0.0223/0.0223 vs 0.0223). *Inference cost:* B 0.57 s → sym. B 2.60 s (4.5×) and D 0.65 → 3.13 s (4.8×) for 1000 videos (eight forward passes, not exactly 8× because small batches do not saturate the GPU); training cost is unchanged for the wrapper, while B itself already costs ≈ 1.6–1.7× the training time per epoch. **When would built-in symmetry hurt?** Averaging eight predictions blurs when the predictions are sharp and the views disagree, i.e. when the world is *not* symmetric: a drift or gravity in one direction, orientation-specific appearance that matters for the forecast, or a symmetry that holds only approximately. It did not hurt here: the dynamics (constant velocity, elastic bounces in a square box) are exactly G-symmetric and the forecasts are already blurry, so averaging acts like a small ensemble (−0.6% to −2.8% error).

Q2.16 Video 9279, $T = R$ (Figures 3 and 6(a)). Model A's two views disagreed (rotated input: stalled blob, centroid 0.27 px/frame vs truth 1.46; original: 1.44). In the final model the disagreement is gone: rows 3 and 4 are identical (centroid gap 0.0 px); but it is *not* right: both views move at 0.79 px/frame (truth 1.46), and the forecast's ink grows from 306 to 480 (truth 255 → 273), i.e. it still under-moves and blurs. Which change fixed what: *symmetrisation* fixed the disagreement (consistency: $C_R = 0$); *symmetry augmentation (B)* reduced the stall in the rotated view (0.27 → 1.19 px/frame, B seed 0) — so the stall was mostly appearance; *speed augmentation and reach (C, D)* did not change this 1× video (D: 0.78/1.11, i.e. no improvement), and the remaining under-motion is the general 1× cost of Q2.14 (D, C are less sharp at 1× than A). In short: consistency repaired, accuracy on this overlap video not.

Q2.17 Final verdict (150 words). *Partly.* Evidence for motion rather than memory: exact G-consistency ($C_M = C_R = 0$, $E_M = E_R = E^{(5)} = 0.0265$); the same error on *time-reversed* videos (0.0264 vs 0.0265, Table S2); at 2×/3× the digits travel 88%/76% of the true distance (A: 39%/22%) and $E^{(2)}$ falls to 0.0282/0.0318 (A 0.0444/0.0516). Against: forecasts stay blurry (0.0265, only 38% below Blank) and under-move at 3×. *A skeptic would be convinced* by sharp forecasts with the right displacement at speeds *outside* the training set (e.g. non-integer or faster). *Test not passed:* tracking digit identity through overlaps — Figure 6a (video 9279) is still blurred and slow.

Generative-AI use disclosure (course AI policy)

Tool. Cursor (Claude) in VS Code, used locally with the assignment folder and conda environment `PA1_DL`.

How it was used. The assistant was used to understand the handout, to ask for section templates, and to sanity-check wording and numbers. Training, evaluation, file layout, and the submitted claims were the student's. Suggested text was kept only after it matched the handout and the saved result files.

Prompts given by the student (15 representative questions; not a request to complete the assignment):

1. "Please explain Assignment 2 in your own words: what is Q1 vs Q2, and what must be in the report vs the GitHub repo?"
2. "Give me a heading template for Question 1 that matches the PDF (architecture, assumptions, task, data, training, robustness). Do not fill the answers."
3. "Which river facts are given, and which ones am I allowed to assume? I want history length and the 60-minute horizon labelled as assumptions if the brief does not fix them."
4. "Is future rainfall allowed as an input? Is an already announced reservoir release allowed? Please quote the distinction."
5. "For Moving MNIST, may I use torchvision's built-in train/test split, or must I split by video index? I am using conda env `PA1_DL`."
6. "What should Table 1 contain, and how are speed and the symmetry group G defined in the handout? A checklist is enough."
7. "How do I count parameters of a ConvLSTM so I can check that Model A is 480,784?"
8. "What does the optical-flow figure need to show (inputs, arrows, calibration)? A caption template only."
9. "Give me a short markdown template for the Q2.5 prediction file. Remind me that it has to be committed before I run the mirror / rotation / speed tests."
10. "Same for Q2.11: a prediction template, and remind me it must be committed before I train B, C, and D."
11. "In the PDF, what is the difference between models A, B, C, and D? I will choose D myself; I only want the definitions."
12. "Write the symmetrised-forecaster formula from the handout so I can check my implementation of C_T and E_T."
13. "Give me a report outline (Q1, Q2a, Q2b, tables, figures) with empty bullets. I will write the content."
14. "Please shorten my Question 1 headings and suggest how to draw the architecture in TikZ. Do not change the design."
15. "My GitHub URL is `https://github.com/ihuzafaone/DLSTG-PA2`, name Huzaifa, roll 25280009. Put those in the header only. Then list remaining PDF requirements I may have missed."

Coding help (summary). Asked for a repo layout, a Table 1 checklist, a parameter-count check for Model A, a caption template for the optical-flow figure, a Q1 section template, a reminder of what Model D is allowed to change, and a report outline. Did not ask the assistant to train models or to invent results.

Help on code. Example snippets and naming suggestions for data loading, ConvLSTM, metrics, training entry points, and symmetrisation (in this repo: `src/mmdata.py`, `src/models.py`, `src/evalkit.py`, `src/train.py`, `src/table1.py`, `src/fig1_flow.py`, `q1/river_prototype.py`). The student adapted those ideas, ran jobs in `PA1_DL` on a local NVIDIA RTX 4060 Laptop GPU (8 GB), and kept Model D's dilation and the D+B run only after validation screening.

My checks on code. Confirmed Table 1 against the handout; confirmed Model A (and the screened Model D) at 480,784 parameters; confirmed that every reported test number matches the JSON files in `results/`.

Question 1. The assistant drafted empty six-heading templates and TikZ layout hints. History length L=36 and the six-step (60 min) horizon are assumptions A1/A2 in the sense that the brief leaves both unspecified; the section keeps those separate from the given river. Future rain is not an input; an already announced release is. The diagram is the TikZ figure; headings were shortened by the student.

Question 2. The assistant explained required tables/figures and commented on scripts. The student ran Table 1, Figure 1, training, and evaluation, and checked the numbers. Q2.5 was written before the mirror, rotation, and speed tests; Q2.11 was written before training B, C, and D.

Output kept from the assistant. Clarifications of what the handout was asking, blank section templates, reminders of formulas already given in the brief, and help tightening wording, plus a checklist used on a final pass. Every number reported here was read off the student's own saved result files, never supplied by the assistant.